Tshwane University of Technology
ISJ107V
Assignment 2

Project Title
AI-Enhanced Community-Based Missing Children Reporting and Tracking System

Submitted by
Ntwanano Mathebula
Student Number: 216080254

Lecturer: Dr HD Masethe
Due Date: 30 June 2026

Table of Contents

Contents

Contents 2
Description/Summary 3
Problem Statement 3
Technology Stack 4
Emerging Tech Integration Goals 5
Justification for Emerging Technology 6
Functional Requirements 7
Non-Functional Requirements 9
Emerging Tech-Specific Requirements 9
User Stories 10
Database Design/Dataset 11
API Design 13
Emerging Tech Code Integration 14
Testing and Evaluation 18

Description/Summary

This project proposes a centralized, web-based platform designed for South African community members to report missing children, share information, and receive real-time alerts about active cases. The idea came from recognizing how scattered the current reporting process is — people post on WhatsApp groups, Facebook pages, or go directly to a police station, and none of these channels communicate with each other. This results in delays, confusion, and duplicated effort at a time when speed matters most.

The platform allows registered users to submit missing children reports with identifying details and optional photographs, search existing reports, and receive notifications when new cases appear in their area. All report data is stored centrally in a PostgreSQL database and accessed through a secure Java Spring Boot backend.

The emerging technology integrated into the system is Artificial Intelligence and Machine Learning (AI/ML) — specifically NLP-based text similarity analysis implemented in Python. The AI module is designed to detect when two reports are likely referring to the same child, by comparing names, physical descriptions, and location data using cosine similarity scoring. A data analytics module was also incorporated to surface reporting patterns across different locations and time periods, which could help community organizations and law enforcement prioritize their responses more effectively.

Problem Statement

South Africa has a serious and ongoing problem with missing children. According to Missing Children South Africa (2024), approximately 1 697 children are reported missing every year — that works out to roughly one child every five hours. What makes this worse is that there is currently no structured digital platform specifically built for communities to report and track these cases.

At the moment, information gets shared through a mix of social media posts, WhatsApp community groups, visits to police stations, and word of mouth. The problem with this approach is that none of these channels are connected. The same case can appear across five different platforms with slightly different details, which causes confusion and wastes the time of people trying to respond.

The specific problems this fragmentation creates include:

- Critical information reaches community members too slowly, reducing the chance of a quick response
- The same child gets reported multiple times across different channels, leading to duplicated effort and conflicting information

- There is no central place to identify patterns — for example, whether certain areas show higher rates of missing children reports during specific months
- There is no structured alert system that notifies people based on where they live

The absence of a unified platform directly reduces the speed and effectiveness of community response, lowering the chances of locating missing children quickly. The AI/ML duplicate detection component was added specifically to address the duplication problem, which came up as one of the most practical and impactful challenges to solve — it is not just an inconvenience, it actively slows down response efforts by fragmenting information that should be consolidated.

Technology Stack

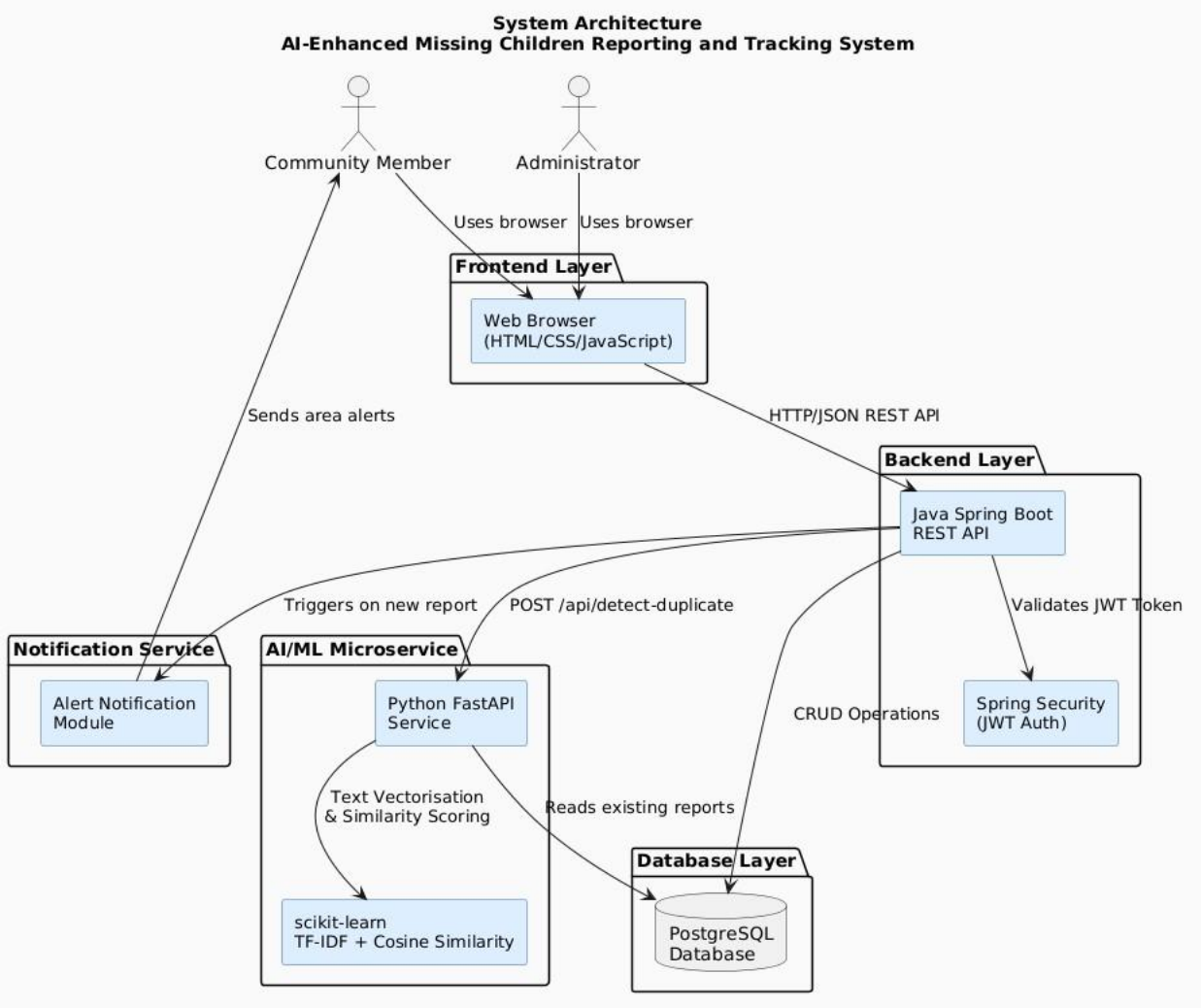
The technology choices for this project were guided by two priorities: using tools that are well-supported and production-ready and keeping the architecture modular so that individual components — particularly the AI module — can be updated independently.

Layer	Technology	Purpose
Backend	Java Spring Boot	REST API, business logic, authentication
Database	PostgreSQL	Persistent storage for users, reports, images
Frontend	HTML, CSS, JavaScript	Web-based user interface
AI/ML Module	Python (Flask/FastAPI)	Duplicate detection via NLP text similarity
NLP Library	scikit-learn / rapidfuzz	TF-IDF vectorisation and cosine similarity
Security	Spring Security + BCrypt	Authentication and password hashing
Version Control	Git	Source code management
Backend IDE	IntelliJ IDEA	Java/Spring Boot development
Frontend IDE	Visual Studio Code	HTML/CSS/JS development
API Communication	REST (HTTP/JSON)	Java backend calls Python AI service

Java Spring Boot was selected for the backend because of its mature ecosystem, built-in support for REST API development, and straightforward integration with Spring Security for authentication. PostgreSQL was chosen over alternatives like MySQL because of its stronger support for complex queries and ACID-compliant transactions, which matter when dealing with sensitive case data.

The Python AI module was deliberately kept as a separate microservice rather than embedded into the Java backend. This separation means the similarity model can be retrained or swapped out without touching the main application. FastAPI was chosen to expose the Python module because it is lightweight, fast, and generates automatic API documentation, which simplifies integration testing.

The frontend was kept intentionally simple — HTML, CSS, and vanilla JavaScript — to avoid adding unnecessary complexity for what is primarily a form-and-search interface.



Emerging Tech Integration Goals

Goal 1 — Duplicate Report Detection

Implement a Python-based NLP similarity analysis module that compares each incoming missing children report against all existing reports and automatically flags potential duplicates when a cosine similarity score of 0.75 or higher is returned. Success will be measured by the system's ability to reduce duplicate entries in the database by at least 60% compared to an unfiltered submission process. This will be evaluated during AI/ML testing using a prepared dataset of known duplicate and non-duplicate report pairs.

Goal 2 — Analytics and Pattern Detection
Develop a data analytics dashboard that aggregates missing children report data by geographic location and time period. The dashboard should allow administrators and community members to identify high-risk areas and seasonal reporting patterns at a glance. Success means the dashboard correctly displays at least three distinct analytics views — reports by location, reports by month, and reports by resolution status — all sourced from live database data.

Goal 3 — Real-Time AI Integration
Integrate the Python AI/ML module with the Java Spring Boot backend via a REST API so that duplicate detection runs automatically on every new report submission. The integration must be seamless from the user's perspective — they submit a report and receive a response, whether that response is a success confirmation or a duplicate warning, within 2 seconds. This latency target accounts for both the Spring Boot processing time and the Python service response time combined.

Goal 4 — Continuous Improvement
Design the AI module so it can be retrained or updated independently of the main web application. As more reports are submitted over time, the vocabulary and patterns in the data will evolve. The module should be structured so that updating the similarity model requires no changes to the Java backend — only the Python service needs to be redeployed. This supports the long-term usefulness of the system beyond the initial submission.

Justification for Emerging Technology

Before committing to NLP-based similarity analysis, I considered whether a simpler approach — like keyword search or exact field matching — would be sufficient to detect duplicate reports. After thinking it through, I concluded it would not work reliably in this context, for several reasons.

Community members do not describe the same child the same way. One person might write "wearing a blue shirt" while another writes "blue top." A name might be spelled differently, or a location described as "near Soshanguve" versus "Soshanguve township." Standard string matching would miss all of these as potential duplicates, and the result would be a database full of duplicates that the system failed to catch.

NLP-based text similarity using TF-IDF vectorization and cosine similarity solves this problem. Instead of comparing strings character by character, the approach converts text into numerical vectors representing

the frequency and importance of terms across the corpus, then measures the angle between those vectors. Two reports describing the same child in slightly different words will produce vectors that are directionally similar — and that similarity will be captured in the cosine score.

This technique has proven applicability in domains like plagiarism detection, record deduplication, and entity matching (Ruiz Reyes et al., 2024), which gave me confidence it was appropriate for this use case. The Python libraries scikit-learn and rapidfuzz were chosen specifically because they are well-documented, actively maintained, and can be exposed as a microservice via FastAPI — making integration with the Java backend straightforward without requiring significant additional infrastructure.

The direct benefit to the community is cleaner, more reliable data. When duplicate reports are caught early and flagged for review, administrators can merge them, and the community sees one accurate, consolidated case rather than multiple conflicting posts across the system.

Functional Requirements

Based on FURPS+ — Functionality

F1 — User Registration and Authentication
Community members must be able to create an account using a valid email address and password. The system must authenticate users through a secure login page before granting access to any report data or submission features.

F2 — Missing Children Report Submission
Once logged in, a user must be able to fill in and submit a missing children report. Required fields include the child's full name, age, gender, physical description, last known location, and date last seen. Attaching a photograph is optional but supported.

F3 — Duplicate Detection
Every time a new report is submitted, the system must automatically run it through the AI similarity module and compare it against all existing reports. If a similarity score of 0.75 or higher is returned, the report must be flagged and queued for administrator review without blocking the submission from being saved.

F4 — Report Search and Viewing
Authenticated users must be able to search through existing active reports. The search must support filtering by child name, last known location, age range, and the date range of when they were last seen.

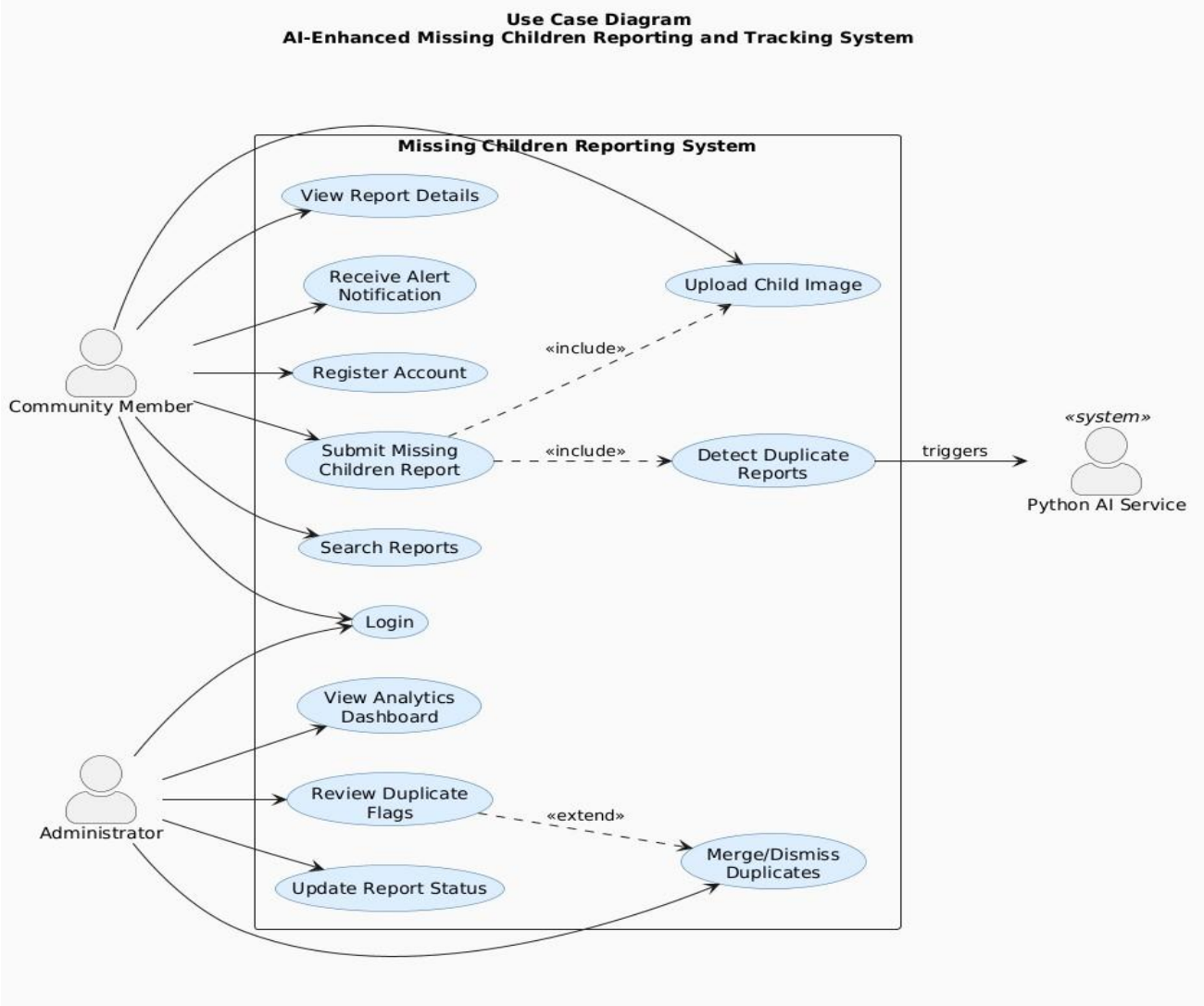
F5 — Community Alert Notifications

When a new report is submitted and confirmed as unique, the system must send an alert notification to registered users whose location matches the report's area. The goal is to get information to relevant community members as quickly as possible.

F6 — Analytics Dashboard
The system must provide a dashboard displaying report statistics broken down by geographic location, time period, and resolution status (active, found, or duplicate). This is intended primarily for administrators and community organizations monitoring trends.

F7 — Administrator Report Management
Administrators must have a dedicated interface to review flagged duplicate reports. They must be able to either merge two reports into one or dismiss the duplicate flag. They must also be able to update a report's status when a child has been found.

F8 — Image Upload
Users must be able to attach a photograph to a missing children report. The system must only accept JPG or PNG files and must enforce a maximum file size of 5MB.



Non-Functional Requirements

Based on FURPS+ — Usability, Reliability, Performance, Security

U1 — Usability
The interface must be intuitive enough that a community member with no technical background can submit a report without confusion. All primary functions — submitting a report, searching cases, and checking notifications — must be reachable within three clicks from the home page.

R1 — Reliability
The system must maintain 99% uptime during the demonstration and evaluation period. Given that missing children's cases are time-sensitive, downtime is not acceptable during active use.

P1 — Performance
The full report submission flow, including the AI duplicate check, must complete within 2 seconds under normal load conditions. This keeps the experience responsive for users and ensures the AI component does not create a noticeable delay.

S1 — Security
All user passwords must be stored as BCrypt hashes — plain text storage is not acceptable. All API endpoints must require a valid JWT token. No personally identifiable information about children may be accessible to unauthenticated users under any circumstances.

S2 — Data Integrity
All database transactions must use ACID-compliant operations. This ensures that a report submission either completes fully or rolls back cleanly — partial data entries must not be possible.

Emerging Tech-Specific Requirements

ET1 — Similarity Threshold
The duplicate detection module must use a cosine similarity threshold of 0.75. Reports scoring at or above this value are flagged as potential duplicates. This threshold was chosen as a practical starting point — high enough to reduce false positives while still catching near-duplicate descriptions with different wording.

ET2 — Inference Latency
The Python AI similarity service must return a similarity score within 500ms per comparison request. This target was set to ensure the service stays within the 2-second total submission budget, leaving adequate time for the Spring Boot processing and database operations.

ET3 — Model Input

The AI module must accept the combined text of three report fields — child name, physical description, and last known location — as a single concatenated input string for vectorization. These three fields together contain the most distinguishing information for identifying duplicate reports.

ET4 — API Contract
The Python AI module must expose a REST endpoint at POST /api/detect-duplicate. It must accept a JSON payload containing the new report text and a list of existing report texts with their IDs. It must return a JSON response containing the highest similarity score and the ID of the matched report, or a null match ID if no score reaches the threshold.

ET5 — Fallback Behaviour
If the Python AI service is unavailable or returns an error, the system must not block the report submission. The report must be saved as active, the AI check failure must be logged to the system log, and the administrator must be notified that the duplicate check did not run for that submission.

User Stories

US1 — Report Submission
As a community member, I want to submit a missing children report so that other community members and responders can be alerted quickly.

Acceptance criteria: The report is saved to the database with all required fields present. The submitting user sees a confirmation message. An alert notification is sent to registered users in the same geographic area within 1 minute of submission.

US2 — Duplicate Detection
As an administrator, I want the system to automatically identify reports that look like duplicates so that I can merge them before they cause confusion.

Acceptance criteria: When a new report is submitted with a similarity score of 0.75 or higher against an existing report, it is automatically flagged in the system. The administrator receives a notification identifying both the new report and the matched existing report, along with the similarity score.

US3 — Alert Notification
As a registered user, I want to receive an alert when a new missing children report is submitted near my location so that I can look out for the child or share the information.

Acceptance criteria: The user receives a notification within 1 minute of a qualifying report being submitted. The notification includes the child's name, approximate age, and last known location.

US4 — Analytics Dashboard
As an administrator, I want to view reporting trends by location and time period so that I can identify areas or seasons with higher rates of missing children reports.

Acceptance criteria: The dashboard displays aggregated report data filterable by location and date range. Data reflects newly submitted reports without requiring manual refresh.

US5 — Report Search
As a community member, I want to search for missing children reports by name or location so that I can quickly find information about a specific case.

Acceptance criteria: Search results are returned within 2 seconds and accurately reflect the entered search criteria. Results can be filtered by name, location, age range, and date range simultaneously.

Database Design/Dataset

The database is structured around four core tables. The central relationship is between users and reports — a user submits one or many reports, and each report belongs to exactly one submitting user. The duplicate_flags table records relationships between reports when the AI module identifies a potential match. The notifications table links users to reports, recording which alert was sent to whom and when.

All primary keys use UUID rather than sequential integers to avoid predictable record IDs, which is a basic security consideration for a system handling sensitive data.

Users Table

Column	Type	Notes
user_id	UUID (PK)	Unique user identifier
full_name	VARCHAR(100)	User's full name
email	VARCHAR(150)	Unique, used for login
password_hash	VARCHAR(255)	BCrypt hashed
role	ENUM(user, admin)	Access control
location	VARCHAR(100)	User's area for alerts
created_at	TIMESTAMP	Registration date

Reports Table

Column	Type	Notes

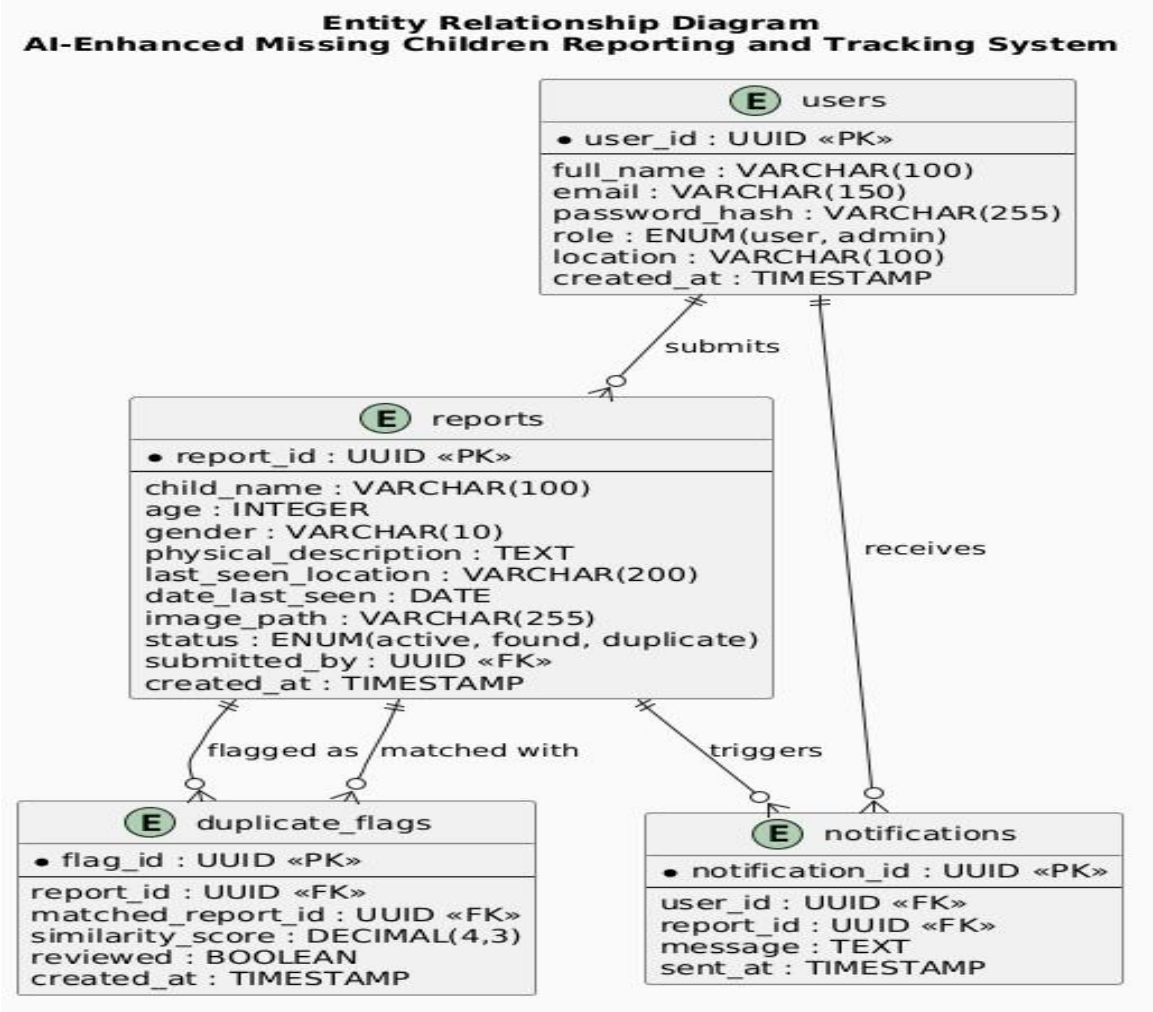
report_id	UUID (PK)	Unique report identifier
child_name	VARCHAR(100)	Missing child's name
age	INTEGER	Child's age
gender	VARCHAR(10)	Child's gender
physical_description	TEXT	Full text description
last_seen_location	VARCHAR(200)	Last known location
date_last_seen	DATE	Date last seen
image_path	VARCHAR(255)	Path to uploaded image
status	ENUM(active, found, duplicate)	Report status
submitted_by	UUID (FK → users)	Who submitted
created_at	TIMESTAMP	Submission timestamp

duplicate_flags Table

Column	Type	Notes
flag_id	UUID (PK)	Unique report identifier
report_id	UUID (FK → reports)	New report flagged
matched_report_id	UUID (FK → reports)	Existing similar report
similarity_score	DECIMAL(4,3)	Score from AI module
reviewed	BOOLEAN	Admin reviewed flag
created_at	TIMESTAMP	Flag creation time

Notifications Table

Column	Type	Notes
notification_id	UUID (PK)	Unique notification record
user_id	UUID (FK → users)	Recipient user
report_id	UUID (FK → reports)	The report that triggered it
message	TEXT	Notification message content
sent_at	TIMESTAMP	When it was sent



API Design

The API follows REST conventions throughout. All endpoints return JSON. Authentication endpoints are the only ones accessible without a token — every other endpoint requires a valid JWT Bearer token in the Authorization header. This was a deliberate decision to ensure no report data is ever accessible to unauthenticated users.

The API is split into four logical groups:

Authentication

- *'POST /api/auth/register'* — Register new user

- '*POST /api/auth/login*' — Authenticate user, return JWT token

Reports

- *'POST /api/reports'* — Submit new missing children report (triggers AI check)
- *'GET /api/reports'* — Get all active reports (with filters: location, age, date)
- *'GET /api/reports/{id}'* — Get specific report details
- *'PUT /api/reports/{id}/status'* — Update report status (admin only)

Duplicate Detection (Python AI Service)

- *'POST /api/detect-duplicate'* — Accepts report text, returns similarity score and matched report ID

Notifications

- *'GET /api/notifications'* — Get notifications for authenticated user

Analytics

- *'GET /api/analytics/trends'* — Returns report counts by location and time period

The duplicate detection endpoint sits on the Python FastAPI service rather than the Spring Boot backend. The Java backend calls it internally after saving each new report — it is not exposed to the frontend directly.

Emerging Tech Code Integration

The Python AI module runs as a separate microservice built with FastAPI, which means it operates independently from the Java Spring Boot backend. The decision to keep them separate was deliberate — if the similarity model needs to be adjusted, retrained, or replaced with a more sophisticated approach later, it can be done without touching the Java code or redeploying the main application.

Duplicate detection flow, step by step:

1. A user fills in the report form and clicks submit

2. The Spring Boot controller receives the POST request, validates the JWT token, and saves the report to PostgreSQL with a status of "active"

3. The backend combines the report's name, physical description, and last known location into a single text string

4. That combined string is sent to the Python service via HTTP POST to `/api/detect-duplicate`

5. The Python service retrieves all existing report texts from PostgreSQL, applies TF-IDF vectorization using scikit-learn, and calculates cosine similarity between the new report and every existing one

6. If the highest similarity score is 0.75 or above, the service returns the matched report ID and score to the Java backend

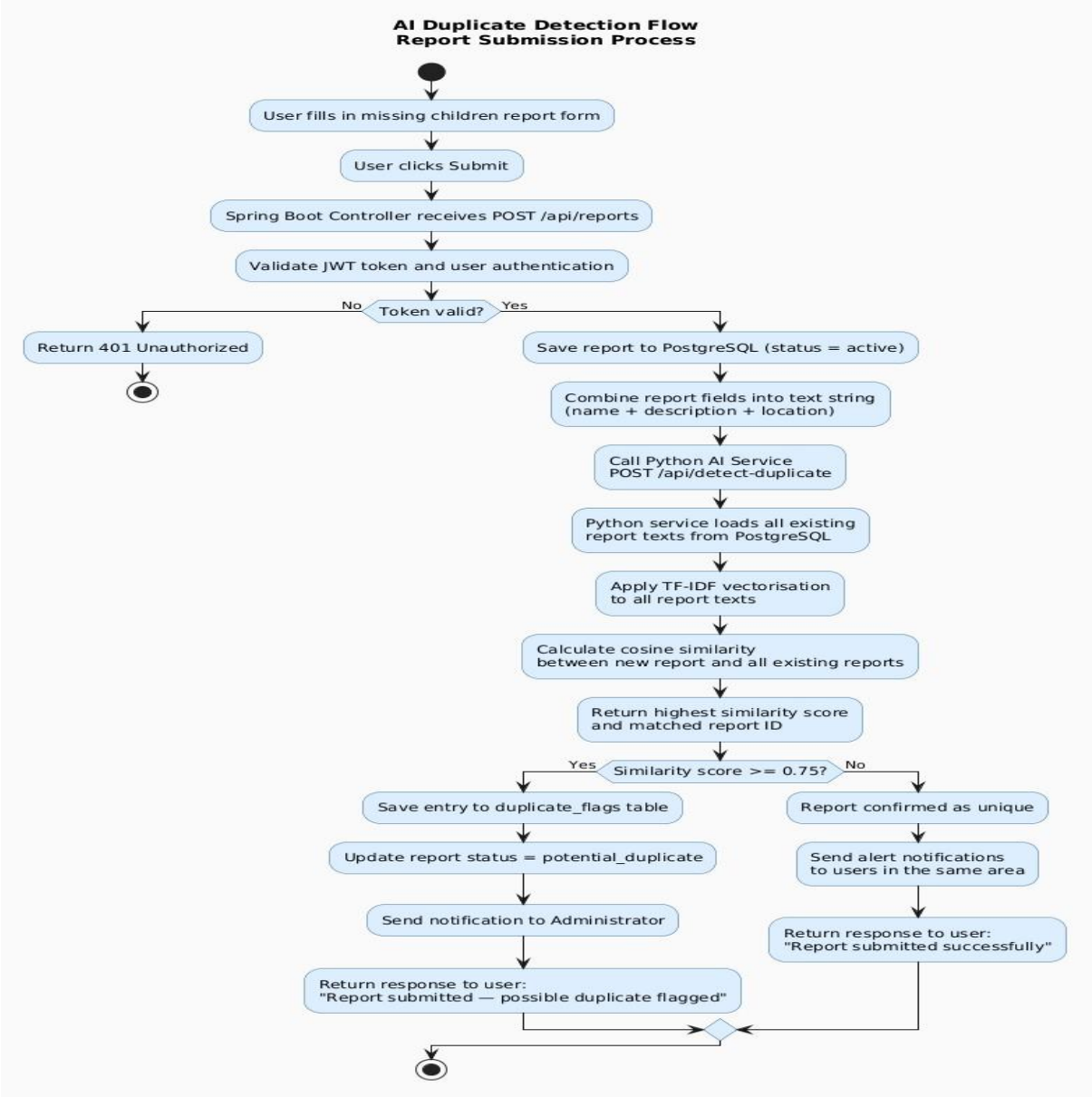
7. Spring Boot saves the flag to the `duplicate_flags` table, updates the report status, and sends a notification to the administrator

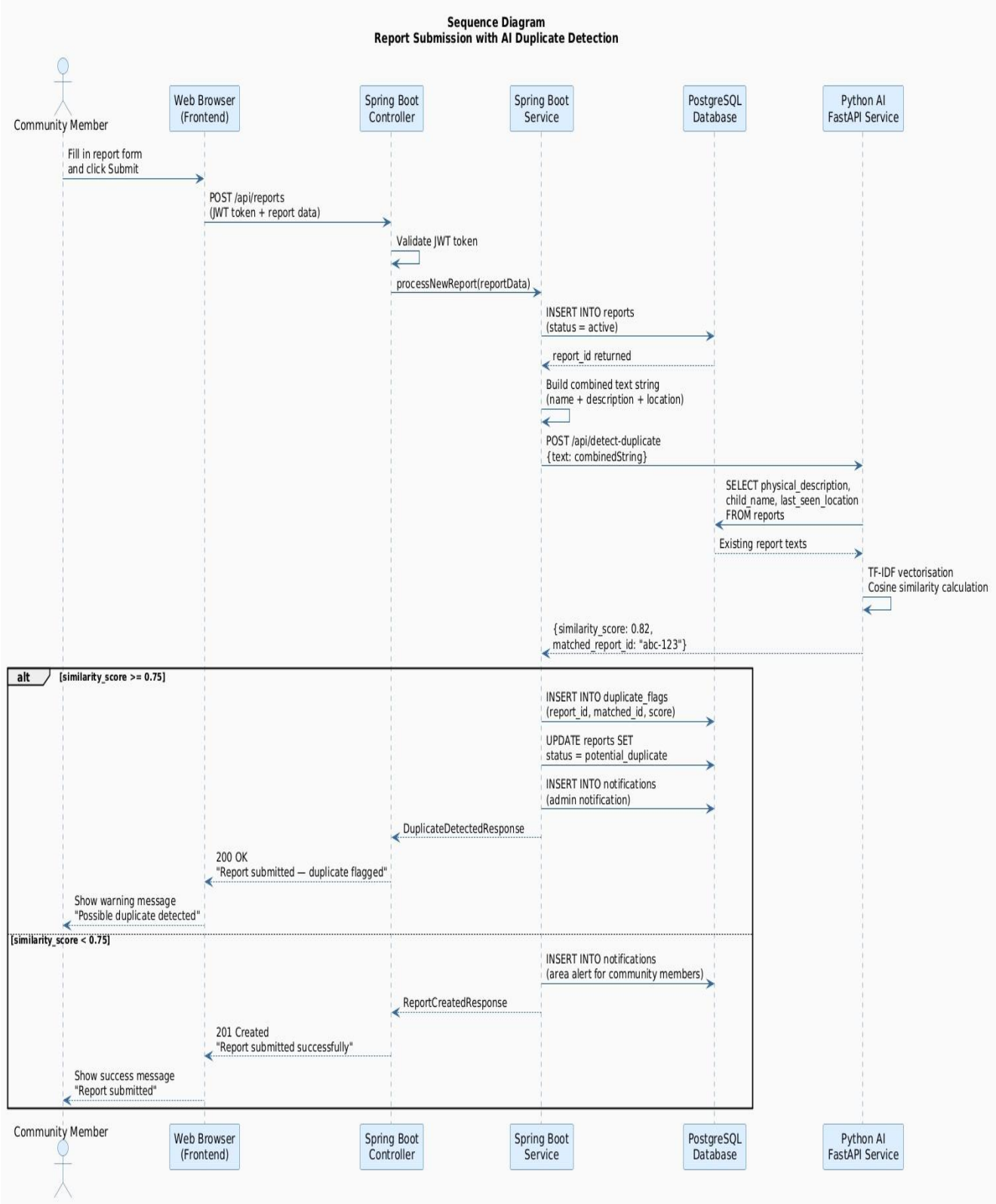
8. If the score is below 0.75, the report is confirmed unique and area alerts are sent to relevant users

The core detection logic in Python:

```
 from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

def detect_duplicate(new_report_text, existing_reports):

    # Combine new report with all existing report texts for joint
vectorization

    all_texts = [new_report_text] + [r['text'] for r in existing_reports]

    vectorizer = TfidfVectorizer()

    tfidf_matrix = vectorizer.fit_transform(all_texts)

    # Compare the new report (row 0) against all existing reports (rows 1
onward)

    similarity_scores = cosine_similarity(

        tfidf_matrix[0:1],

        tfidf_matrix[1:]

    )

    return similarity_scores
```

The threshold of 0.75 was chosen as a starting point based on common practice in text deduplication tasks. If testing reveals too many false positives or missed duplicates, this value can be adjusted without changing any other part of the code.


AI Duplicate Detection Flow
Report Submission Process
User fills in missing children report form
User clicks Submit
Spring Boot Controller receives POST /api/reports
Validate JWT token and user authentication
Token valid?
No
Yes
Return 401 Unauthorized
Save report to PostgreSQL (status = active)
Combine report fields into text string (name + description + location)
Call Python AI Service POST /api/detect-duplicate
Python service loads all existing report texts from PostgreSQL
Apply TF-IDF vectorisation to all report texts
Calculate cosine similarity between new report and all existing reports
Return highest similarity score and matched report ID
Similarity score >= 0.75?
Yes
No
Save entry to duplicate_flags table
Update report status = potential_duplicate
Send notification to Administrator
Return response to user: "Report submitted — possible duplicate flagged"
Report confirmed as unique
Send alert notifications to users in the same area
Return response to user: "Report submitted successfully"


Sequence Diagram
Report Submission with AI Duplicate Detection
Community Member
Web Browser (Frontend)
Spring Boot Controller
Spring Boot Service
PostgreSQL Database
Python AI FastAPI Service
Fill in report form and click Submit
POST /api/reports (JWT token + report data)
Validate JWT token
processNewReport(reportData)
INSERT INTO reports (status = active)
report_id returned
Build combined text string (name + description + location)
POST /api/detect-duplicate {text: combinedString}
SELECT physical_description, child_name, last_seen_location FROM reports
Existing report texts
TF-IDF vectorisation Cosine similarity calculation
{similarity_score: 0.82, matched_report_id: "abc-123"}
alt
[similarity_score >= 0.75]
INSERT INTO duplicate_flags (report_id, matched_id, score)
UPDATE reports SET status = potential_duplicate
INSERT INTO notifications (admin notification)
DuplicateDetectedResponse
200 OK "Report submitted — duplicate flagged"
Show warning message "Possible duplicate detected"
[similarity_score < 0.75]
INSERT INTO notifications (area alert for community members)
ReportCreatedResponse
201 Created "Report submitted successfully"
Show success message "Report submitted"
Community Member
Web Browser (Frontend)
Spring Boot Controller
Spring Boot Service
PostgreSQL Database
Python AI FastAPI Service

Testing and Evaluation

Unit Testing

The duplicate detection function will be tested using pairs of reports that I have manually prepared — some that clearly describe the same child with slightly different wording, and others that describe entirely different cases. The purpose is to verify that similarity scores fall within expected ranges before the module is integrated with the backend. Each API endpoint will also be tested individually to confirm it returns the correct HTTP status codes and response structures.

Integration Testing

The most important test is the full end-to-end submission flow: a report submitted through the web form should travel through Spring Boot, be saved to PostgreSQL, trigger the Python AI check, and produce a duplicate flag entry where applicable. Notification delivery will be verified as part of this — alerts should reach users within 1 minute of a successful unique submission. JWT authentication will be tested across all protected endpoints to confirm that missing or invalid tokens are rejected with a 401 response.

AI/ML Evaluation

A test dataset of 20 report pairs will be prepared — 10 confirmed duplicates and 10 confirmed non-duplicates. Running these through the model at the 0.75 threshold will produce a precision and recall score. The targets are precision of at least 80% and recall of at least 75%. If either falls short, I will consider adjusting the threshold or revisiting how the combined input string is constructed before vectorization, since the quality of the input text significantly affects similarity scores.

Performance Testing

The full report submission cycle, including the AI check, must complete within 2 seconds. The Python service on its own should respond within 500ms. These will be tested by timing requests against a populated database under normal load conditions.

Security Testing

Any request to a protected endpoint without a valid JWT must return a 401 Unauthorized response. Passwords must be confirmed as BCrypt hashes in the database — not plain text. Image uploads must be rejected for any file type other than JPG or PNG, and for files exceeding 5MB.

User Acceptance Testing

At least three people will be asked to go through the system — registering an account, submitting a report, and confirming whether they received an alert notification. A separate administrator account will be used to test the duplicate review and merge workflow from end to end.

References

Missing Children South Africa*.** (2024). ***Missing children's statistics.

Missing Children South Africa. Retrieved from https://www.missingchildren.org.za

Ruiz Reyes, N., ***Morales-Bueno***, R., & ***Fdez-Riverola***, F. (2024).

Text similarity techniques applied to record deduplication and

entity matching***. Journal of Information Science***, 50(2), 112–128.